

## Web Applications – Exam #1 (deadline 2026-06-18 at 23:59)

# "Theater"

FINAL version - Modifications highlighted with RED text.

Design and implement a web application to reserve seats at a theater. The theater shape is not specified but each seat can be uniquely identified by a row label (a letter from A to Z) and a number within the row. Seats have two categories: normal and premium. Premium ones are in the first 2 rows. For simplicity there is only one theater with at least 100 seats, and each row must have at least 8 seats. Each row can potentially have a different number of seats.

Reservations may include more than one seat. They are possible using two different mechanisms: direct selection on the map layout, and assignment by specifying the desired number of seats and the category which must be the same for all requested seats. In the latter case, whenever enough seats are available in the same row, regardless of their position, they must be assigned in the same row. Otherwise, any seat assignment (i.e., any position, provided that the category is the requested one) is acceptable.

The design of the user interface is up to the student. It is expected that all choices are made carefully by considering security and reasonable usability. However, the following specifications must be satisfied. It must be possible to:

- see a map of the theater showing the current booking status for each seat using colors. The map must be accessible also to non-registered users.
- have a place where users can manage their reservations. In particular, it is possible: 1) to see on the map each one of the confirmed reservations; 2) to modify reservations by adding and/or removing seats through the map interface; 3) to delete any reservation at once.

A special category of users, named admin, has the possibility to perform the same actions as a normal user but it can also act, with the same actions, on reservations owned by any other user in the system, only to modify and delete them. Admin users cannot create reservations on behalf of other users. Admin users must choose, at login, if they also want to be able to act as admin: in this case it is required that the login operation should include the 2FA procedure with a TOTP.

Finally, the reservation system should prevent a user from re-booking, within 40 seconds, any seat that the same user had previously reserved and then released.

In all cases, after the reservation action, suitable error messages should be shown to the user to explain the result of the operation, either success or failure, and the reason leading to the failure, e.g., not enough seats of type X, seat Y and Z already reserved by another user, too early to re-reserve seat A, etc.

The organization of these specifications into different screens (and potentially different routes), when not specified, is left to the student and is subject to evaluation.

For the 2FA procedure, for simplicity, use the following secret for all users that require it: LXBSMDTMSP2I5XFXIYRGFVWSFI (it is the same used during lectures and labs).

NB: Very important: using the same secret for all users is just for simplicity, the secret has to be stored separately, one for each user that requires 2FA, in the database (as seen during lecture examples).

## Project requirements

- User authentication (login and logout) and API access must be implemented with `passport.js` and **session cookies**, using the mechanisms explained during the lectures. Session information must be stored on the server, as done during the lectures. The credentials must be stored in hashed and salted form, as done during the lectures. **Failure to follow this pattern will automatically lead to exam failure.**
- The communication between client and server must follow the multiple-server pattern, by properly configuring CORS, and React must run in “development” mode with Strict Mode activated. **Failure to follow this pattern will automatically lead to exam failure.**
- The project must be implemented as a **React application** that interacts with an HTTP API implemented in Node+Express. The database must be stored in an SQLite file. **Failure to follow this pattern will automatically lead to exam failure.**
- 2FA verification must be implemented using a TOTP and by using the libraries explained during the lectures, and with the secret specified in the exam text. **Failure to follow this pattern will automatically lead to exam failure.**
- The evaluation of the project will be carried out by navigating the application, **using the starting URL `http://localhost:5173`**. Neither the behavior of the “refresh” button, nor the manual entering of a URL (except /) will be tested, and their behavior is not specified. Also, the application should never “reload” itself as a consequence of normal user operations.
- The user registration procedure is not requested *unless specifically required in the text*.
- The application architecture and source code must be developed by adopting the best practices in software development, in particular those relevant to single-page applications (SPA) using React and HTTP APIs.
- Particular attention must be paid to ensure server APIs are duly protected from performing unwanted, inconsistent, and unauthorized operations.
- The root directory of the project must contain a README.md file and have the same subdirectories as the template project (client and server). The project must start by running these commands: “`cd server; nodemon index.mjs`” and “`cd client; npm run dev`”. A template for the project directories is already available in the exam repository. Do not move or rename such directories. You may assume that nodemon is globally installed.
- The whole project must be submitted on GitHub in the repository created by GitHub Classroom specifically for this exam.
- The project **must not include** the `node_modules` directories. They will be re-created by running the “`npm ci`” command, right after “`git clone`”.
- The project **must include** the `package-lock.json` files for each of the folders (client and server).
- The project may use popular and commonly adopted libraries (for example `dayjs`, `react-bootstrap`, etc.), if applicable and useful. Such libraries must be correctly declared in the `package.json` and `package-lock.json` files, so that the `npm ci` command can download and install all of them.

## Database requirements

- The database schema must be designed and decided by the student, and it is part of the evaluation.
- The database must be implemented by the student, and must be pre-loaded with the information for a theater with at least 4 rows and with at least 3 rows of different length. There must be at least 4 users, and exactly two of them must be able to act as admin. There must be at least 4 reservations, owned by exactly 2 users: one user who can act as admin and one regular user. There must be at least 20 non-reserved seats in the theater.

## Quality requirements

In addition to the implementation of the required application functionality, the following quality requirements will be evaluated, as usual:

- Database design and organization
- Design of the HTTP APIs.
- Organization of React components and routes.
- Correct usage of React patterns (functional behavior, hooks, state, context, and effects).
- Avoiding direct manipulation of the DOM or browser behaviors outside React.
- Code clarity.
- Absence of errors (and warnings) in the client console (except those caused by errors in the imported libraries and by HTTP status different from 2xx).
- Absence of application crashes or unhandled exceptions.
- Basic usability and user-friendliness.
- Originality of the solution.

## Contents of the README.md file

The README.md file must contain the following information (a template is available in the project repository). The clarity and correctness of this file is also part of the evaluation. Generally, each information should take no more than 1-2 lines.

1. Server-side:
  - a. A list of the HTTP APIs offered by the server, with a short description of the parameters and of the exchanged objects.
  - b. A list of the database tables, with their purpose, and the name of the columns.
2. Client-side:
  - a. A list of 'routes' for the React application, with a short description of the purpose of each route.
  - b. A list of the main React components developed for the project. Minor ones can be skipped.
3. Overall:
  - a. A screenshot of, at least, the **seat selection page on the map**. The screenshot must be embedded in the README by linking the image committed in the repository.
  - b. Usernames and passwords of all the users in the database.

## Submission procedure (IMPORTANT!)

To correctly submit the project, you must:

- **Be enrolled** in the exam call.
- **Accept the invitation** on GitHub Classroom, **using the link specific for this exam**, and correctly **associate** your GitHub username with your student ID.
- **Push the project** in the **branch named “main”** of the repository created for you by GitHub Classroom. The last commit (the one you wish to be evaluated) **must be tagged with the tag final** (note: final is all-lowercase, with no whitespaces, and it is a git ‘tag’, NOT a ‘commit message’), **otherwise the submission will not be evaluated**.

Note: to tag a commit, you may use (from the terminal) the following commands:

```
# ensure the latest version is committed
git commit -m "...comment..."
git push

# add the 'final' tag and push it
git tag final
git push origin --tags
```

NB: the tag name is “final”, all lowercase, no quotes, no whitespaces, no other characters, and it must be associated with the commit to be evaluated.

Alternatively, you may insert the tag from GitHub’s web interface (In section ‘Releases’ follow the link ‘Create a new release’).

Finally, check that everything you expect to be submitted shows up in the GitHub web interface: if it is not there, it has not been submitted correctly.

To test your submission, these are the exact commands that the teachers will use to download and run the project. You may wish to test them in an empty directory:

```
git clone ...yourCloneURL...
cd ...yourProjectDir...
git pull origin main # just in case the default branch is not main
git checkout -b evaluation final # check out the version tagged with
'final' and create a new branch 'evaluation'
(cd client ; npm ci; npm run dev)
(cd server ; npm ci; nodemon index.mjs)
```

Make sure that all the needed packages are downloaded by the `npm ci` commands. Be careful: if some packages are installed globally, on your computer, they might not be listed as dependencies. Always check it in a clean installation (for instance, in an empty Virtual Machine).

The project will be **tested under Linux**: be aware that Linux is **case-sensitive for file names**, while Windows and macOS are not. Double-check the upper/lowercase characters, especially of `import` and `require()` statements, and of any filename to be loaded by the application (e.g., the database).